Use one format for one paper review

Institute: PSIT Kanpur   Branch: IT3   Rollno: 2401640130005

Paper downloaded from: Arxiv

PAPER REVIEWED Sr.No. 3

PROJECT TITLE: OmniRAG : AN INTELLIGENT MULTIAGENT KNOWLEDGE & PRODUCTIVITY PLATFORM

GUIDE NAME: ______________________

DATE OF SUBMISSION: ______________________

---

1. Title of paper: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

2. Author/s: Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela (Facebook AI Research, University College London, New York University)

3. Year of publication: 2020

4. Journal name/ conference name: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, pp. 9459-9474

4.1 Abstract of paper:

Large pretrained language models store a great deal of factual knowledge inside their parameters, but they cannot easily access, inspect, or update that knowledge, which limits their performance on tasks that depend on precise, current facts. The authors propose Retrieval-Augmented Generation (RAG), which pairs a pretrained sequence-to-sequence generator with a non-parametric memory: a dense vector index over Wikipedia accessed through a learned neural retriever. Two formulations are compared, one that conditions the entire generated output on the same retrieved passages and one that can draw a different passage for each output token. Fine-tuning the combined retriever-generator end to end on a range of knowledge-intensive tasks produces new state-of-the-art results on several open-domain question-answering benchmarks and yields generated text that is more specific, more varied, and more factually grounded than a purely parametric baseline.

4.2 Summary of paper:

This paper introduces RAG as a general-purpose fine-tuning approach that blends a parametric memory (a pretrained BART generator) with a non-parametric memory (a dense Wikipedia index searched with a Dense Passage Retriever). Two variants are proposed: RAG-Sequence, which uses one retrieved document to generate the full output sequence, and RAG-Token, which can mix evidence from different documents at each generation step, letting the model combine content from several sources within a single answer.

The retriever and generator are trained jointly, without requiring the model to be told which documents contain the correct answer, so the system learns to retrieve useful evidence purely from downstream task performance. The authors evaluate RAG on open-domain question answering (Natural Questions, TriviaQA, WebQuestions, CuratedTrec), as well as on abstractive question answering, Jeopardy-style question generation, and fact verification (FEVER).

Across these tasks, RAG outperforms both purely parametric closed-book models and prior extractive or retrieve-and-extract pipelines, while producing answers that are judged more specific and factual by human evaluators, and that hallucinate less than closed-book generation. The paper concludes that combining parametric and non-parametric memory in a single differentiable model is an effective and flexible way to ground language generation in external knowledge that can, in principle, be updated by swapping the retrieval index.

5. Key points

5.1 Points that are covered in paper

- Motivation: parametric-only language models struggle to access, inspect, and precisely update factual knowledge.
- The RAG architecture, combining a pretrained seq2seq generator with a dense, retrievable Wikipedia index.
- Two model formulations — RAG-Sequence and RAG-Token — and how they differ in how retrieved passages are used during generation.
- End-to-end joint training of the retriever and generator without explicit supervision on which passage is correct.
- Evaluation across open-domain QA, question generation, and fact-verification benchmarks, with comparisons against extractive and closed-book baselines.

5.2 Points that are not covered in paper

- Coordination among multiple specialized agents — RAG uses a single retriever-generator pipeline rather than distinct planning, retrieval, computation, and verification roles.
- Hierarchical or multi-level indexing for long, structurally rich documents; retrieval operates over flat Wikipedia passages rather than sections within a single long document.
- Numerical or code-based reasoning — there is no dedicated computation or code-execution component.
- Handling of non-text content such as tables or images within retrieved evidence.
- Persistent memory or caching of previously retrieved content across follow-up queries.

5.3 Scope for work:

- Extend the single retrieve-then-generate step into a multi-agent pipeline with dedicated planning, retrieval-verification, and answer-composition agents, in the direction OmniRAG's InfoAgent and SummarizerAgent take.
- Replace flat passage retrieval with hierarchical, section-aware indexing suited to long structured documents such as research papers or financial filings.
- Add explicit multi-hop reasoning so evidence from several non-adjacent sections can be chained together rather than merged implicitly at the token level.
- Integrate a numerical or code-execution module for queries that require calculation rather than pure text generation.
- Explore techniques for keeping the retrieval index current in domains where the underlying documents change frequently.

Guide Remark